

PROJECT PROPOSAL

1. Introduction

Baakhapaa is a Nepali storytelling and gamified engagement platform. It produces original content for a growing audience. As that audience grows, so does the pressure to produce content faster and with greater consistency. The current production process is entirely manual. There is no system to guide the writing, plan the visuals, or coordinate the team.

This proposal presents a practical solution to that problem. An AI-powered pre-production intelligence system built as a web application that takes a creator from a raw idea to a complete production-ready package. The system works through three modules: screenplay generation guided by AI, automated storyboard creation from the finalized script, and team collaboration with version control and one-click export.

What separates this system from every existing tool is the approach. Final Draft, Celtx, and StudioBinder all give the creator a blank page with correct formatting. They are empty containers. A professional screenwriter knows how to fill them. A creator who is still learning does not. This system does not assume prior knowledge. It guides the user through every decision, from genre and structure to individual scene dialogue, using AI at each step.

The system will be built by a single developer using an AI-assisted coding workflow combining foundational programming skills with professional tools including Claude AI and GitHub Copilot. This is standard practice in modern software teams and directly reduces build time without compromising quality. All infrastructure and API costs will be funded by Baakhapaa throughout the development period.

1.1 Existing Screenwriting Software and the Gap

Four major tools currently define the screenwriting software market. Each has significant limitations that this system addresses directly.

Software	What It Does	Key Limitation	Gap This System Fills
Final Draft	Industry standard screenplay formatting. Used by major studios.	No AI guidance. No structure help. One time purchase at \$249.	Final Draft gives a blank page. This system gives a structured framework before the user writes a single line.
Celtx	Cloud based writing and pre production suite with storyboarding and scheduling.	Provides tools but does not guide the writing process. No AI writing assistance.	Celtx manages production after writing. This system helps during writing, with AI generating and improving content in real time.
StudioBinder	Production management platform with scriptwriting added later.	Built for producers, not writers. Writing is secondary. Paid plans start at \$29 per month.	StudioBinder organizes shoots. This system creates the content that gets shot, with AI embedded throughout the creation process.
Arc Studio Pro	Modern screenwriting tool with free tier and collaboration features.	No AI content generation. No storyboard module. Limited to screenplay format only.	Arc Studio formats scripts cleanly. This system generates, structures, and visually plans them with AI at every stage.

Every existing tool assumes the creator already knows what to write. They format it, organize it, and export it. But they do not help create it. This system does. It is not a formatting tool. It is a guided creation environment where AI does the thinking alongside the user.

2. Problem Statement

Every stage of content production at Baakhapaa currently depends on manual effort. This creates five specific problems that slow output, reduce consistency, and limit the platform as it scales:

1. Scripts are written without structure. There is no system to define genre, set act structure, or allocate time across scenes. Every script starts from zero.
2. Visual planning is disconnected from writing. Storyboards are created separately after scripts are done, duplicating effort and creating version mismatches.
3. There is no collaboration infrastructure. Team members cannot edit the same document simultaneously, and there is no version history.
4. The platform has no memory of its own content style. Every new script ignores the tone and voice that defines Baakhapaa as a brand.
5. There is no standardized output format. Scripts and visuals are delivered inconsistently, requiring repeated reformatting before each production.

3. Objectives

1. To develop an AI-powered screenwriting engine that guides creators through genre definition, three-act structure with equal 33% time allocation per act, scene breakdown, and bilingual writing assistance in Nepali and English.
2. To build an automated storyboard module that converts finalized scripts into visual frames with shot type and camera notes for each scene.
3. To deliver a web-based platform with real-time collaboration, version history, and one-click export of production-ready documents.

4. Research Methodology

4.1 Requirement Identification

Requirements were gathered through direct discussion with the Baakhapaa CEO and a review of the existing content production workflow. Functional requirements include screenplay generation, storyboard automation, bilingual support, collaboration, version control, and subscription access. Nonfunctional requirements include browser performance, data security, and system scalability.

4.2 Functional Requirements

No	Requirement	Description	Priority
FR 01	Genre and Parameter Input	User defines genre, tone, target audience, duration, and language before writing begins	High
FR 02	Three Act Structure Generation	System generates a three act framework with 33% time allocation per act based on total duration input	High

FR 03	Major and Minor Scene Breakdown	System divides each act into major turning point scenes and minor transitional scenes with individual time allocations	High
FR 04	AI Script Generation	User provides a scene brief and AI generates full scene content including action lines and dialogue	High
FR 05	AI Improvement Mode	User selects any script section and requests AI to rewrite, shorten, translate, or improve it	High
FR 06	Bilingual Output	System generates and exports content in Nepali, English, or both languages simultaneously	High
FR 07	Script Review and Finalization	Automated review checks scene timing, character name consistency, and act balance before finalization	Medium
FR 08	Storyboard Generation	AI generates one visual frame per scene from finalized script with shot type and camera notes	High
FR 09	Storyboard Controls	User can regenerate individual frames, override shot types, add camera movement notes, and reorder frames	Medium
FR 10	Real Time Collaboration	Multiple users can edit the same script simultaneously with visible cursors and inline comments	High
FR 11	Version History	Every save creates a timestamped restore point with diff view between any two versions	Medium
FR 12	Role Based Access	Admin, Editor, and Viewer roles with defined permissions across all system features	Medium
FR 13	Export System	One click export of script as PDF or Word, and combined production package as single PDF	High
FR 14	Subscription Management	Free, Creator, and Pro subscription tiers with payment integration for external users	Medium

4.3 Non-Functional Requirements

No	Requirement	Description	Category
NF R0 1	Performance	AI script generation responds within 5 seconds. Page loads complete within 2 seconds on standard connections.	Performance

NF R0 2	Availability	System maintains 99% uptime. Scheduled maintenance outside peak usage hours only.	Reliability
NF R0 3	Security	All user data encrypted at rest and in transit. Authentication required for all system access.	Security
NF R0 4	Scalability	System architecture supports growth from 10 to 10,000 users without significant infrastructure changes.	Scalability
NF R0 5	Browser Compatibility	Full functionality on Chrome, Firefox, Safari, and Edge. No installation required.	Usability
NF R0 6	Usability	New users complete first script generation without external guidance. Interface requires no technical knowledge.	Usability
NF R0 7	Data Integrity	No data loss on session timeout or connection failure. Auto save triggers every 30 seconds.	Reliability
NF R0 8	Maintainability	Codebase documented for handover. New features can be added without breaking existing modules.	Maintainability

4.4 Feasibility Study

Technical Feasibility

All technologies selected are production-ready and commercially available. AI APIs from Anthropic and OpenAI are well documented with reliable uptime. FastAPI and React are industry standard frameworks. The developer has foundational Python and web development skills, supplemented throughout by AI-assisted coding tools.

Economic Feasibility

Monthly infrastructure costs are estimated between \$75 and \$175, fully funded by Baakhapaa. This includes all API costs, Claude Pro developer subscription, and hosting. Post launch subscription revenue is projected to offset operational costs within three months.

Cost Item	Monthly Estimate	Notes
Anthropic Claude API	\$20 to \$50	Script generation and AI features
Claude Pro Subscription	\$100	Developer AI coding tool
DALL-E Image API	\$15 to \$40	Storyboard frame generation
Hosting (Vercel and Railway)	\$0 to \$20	Free tiers during development

Database (Supabase)	\$0 to \$25	Free tier covers development phase
Total Infrastructure	\$135 to \$235/month	Funded by Baakhapaa
Developer Compensation	NPR 42,000/month	Negotiable. Full time in office. AI Systems Developer role.

Developer compensation is set at NPR 42,000 per month for a full time in office engagement. This reflects the specialized AI development skill set required, the scope of deliverables, and the direct commercial value of the system being built for Baakhapaa.

Legal Feasibility

All APIs and frameworks operate under commercial licenses permitting production use. No restricted datasets are used. Terms of service and privacy policy will be in place before public launch.

Operational Feasibility

The system is designed for everyday use without any technical background. A creator opens the browser, starts a project, and the system guides them from there. The CEO can add or remove team members and control access levels without any developer involvement.

Schedule Feasibility

The full system is planned for delivery across 12 weeks. Each week ends with a working, testable deliverable. Progress is visible at every stage. The use of AI-assisted coding tools makes this timeline achievable for a single developer without cutting corners.

4.5 Data Collection Strategy

- * Internet Sources: Official documentation for the Anthropic API, OpenAI API, LangChain, FastAPI, and React.
- * Open Data and Kaggle: Public screenplay datasets were used to test and improve the screenwriting engine during development.
- * Interview: Direct sessions with the Baakhapaa CEO to validate requirements throughout development.
- * Peer Review: Final system reviewed by at least two independent content creators before handover.

5. Literature Review

The following works form the academic and technical foundation of this project:

1. Brown et al. (2020) demonstrated that large language models perform structured creative writing tasks effectively with minimal task specific training, supporting the screenplay generation approach in this system.
2. Radford et al. (2019) established the generative capability of transformer models for long form structured text, directly relevant to the script generation module.
3. Ramesh et al. (2021) provided the theoretical basis for generating visual storyboard frames from text descriptions, underpinning the automated storyboarding component.

4. Chase (2022) documented the LangChain framework architecture and multi step agent capabilities. LangChain is the primary AI orchestration framework used in this project.
5. Anthropic (2024) provides official technical documentation for the Claude API, the primary language model used for script generation and AI improvement features.
6. OpenAI (2023) provides benchmarks for GPT-4, used as a reference model for evaluating bilingual performance in script generation.
7. FastAPI (2023) provides official documentation for the backend framework, supporting its selection over Flask and Django.
8. Supabase (2023) documents the database and authentication architecture used for real time collaboration and version control features.
9. Vercel (2024) provides technical justification for the frontend hosting choice including zero configuration deployment.
10. Eloundou et al. (2023) contextualize the impact of AI tools on creative knowledge work, supporting the case for AI assisted pre production systems.

6. Proposed Tools

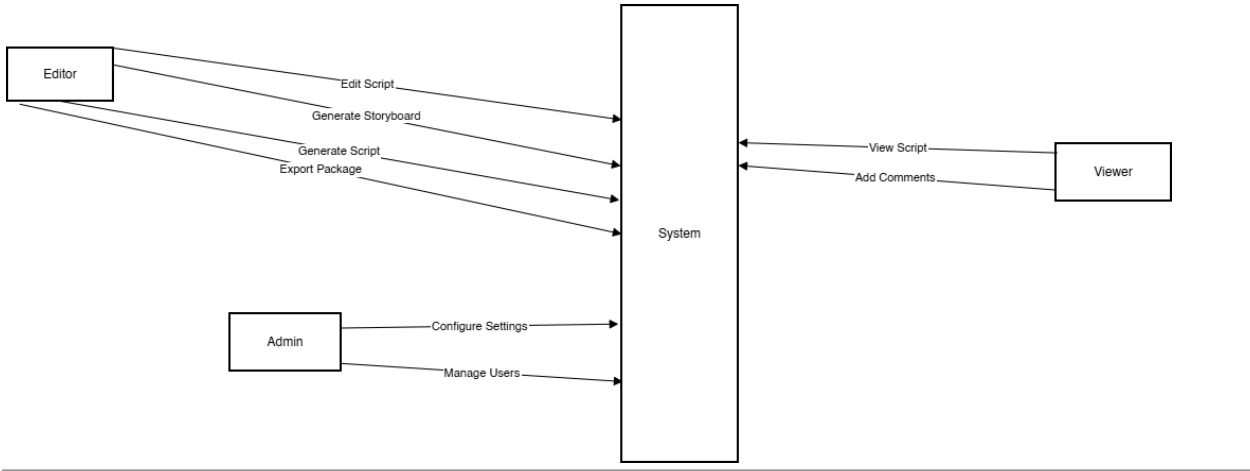
Tool	Category	Reason for Selection
Anthropic Claude API	AI Engine	Best performance for structured bilingual creative writing. Used for all script generation and AI improvement features. Claude Pro subscription included as a direct project development cost.
DALL-E API	Image Generation	Reliable text to image generation from scene descriptions with consistent cinematic outputs suited to storyboard frames.
LangChain	AI Orchestration	Industry standard for chaining multiple AI tasks. Required for multi step workflows such as generating a script and extracting scene data for storyboard generation.
Python with FastAPI	Backend	Python is the dominant language for AI application development. FastAPI provides high performance asynchronous handling and automatic API documentation with minimal code.
React.js with Tailwind CSS	Frontend	Most widely adopted frontend library for interactive web applications. Tailwind enables rapid professional UI development, significantly reducing build time.
Supabase	Database and Auth	Open source PostgreSQL platform with built in real time subscriptions for collaboration and a built in authentication system. Free tier covers the full development phase.
n8n	Automation	Open source and self hostable with no per task pricing. Used to connect services and trigger workflows. More cost effective than Zapier or Make for this scope.
Vercel and Railway	Hosting	Vercel is standard for React frontend deployment with zero configuration. Railway handles Python backend hosting. Both offer free starter plans suitable for development.
Claude Pro and GitHub Copilot	Dev Tools	AI assisted coding tools used throughout development to accelerate implementation. Standard in modern professional software teams. All code is reviewed, tested, and owned by the developer.

7. System Diagrams

The diagrams below define the structure, data model, and behavior of the system. Mermaid code for each diagram is provided separately for rendering. Each labeled space below is reserved for the corresponding diagram to be inserted once generated.

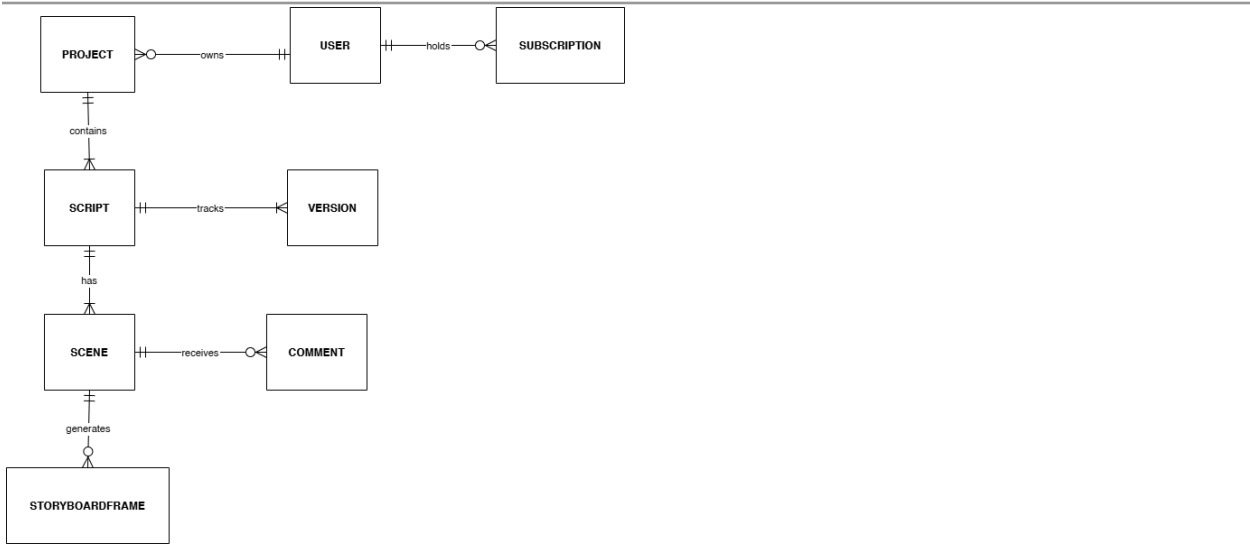
7.1 Use Case Diagram

It shows the interactions between the three user roles (Admin, Editor, and Viewer) and all system functions, including script generation, storyboard creation, collaboration, version control, and export.



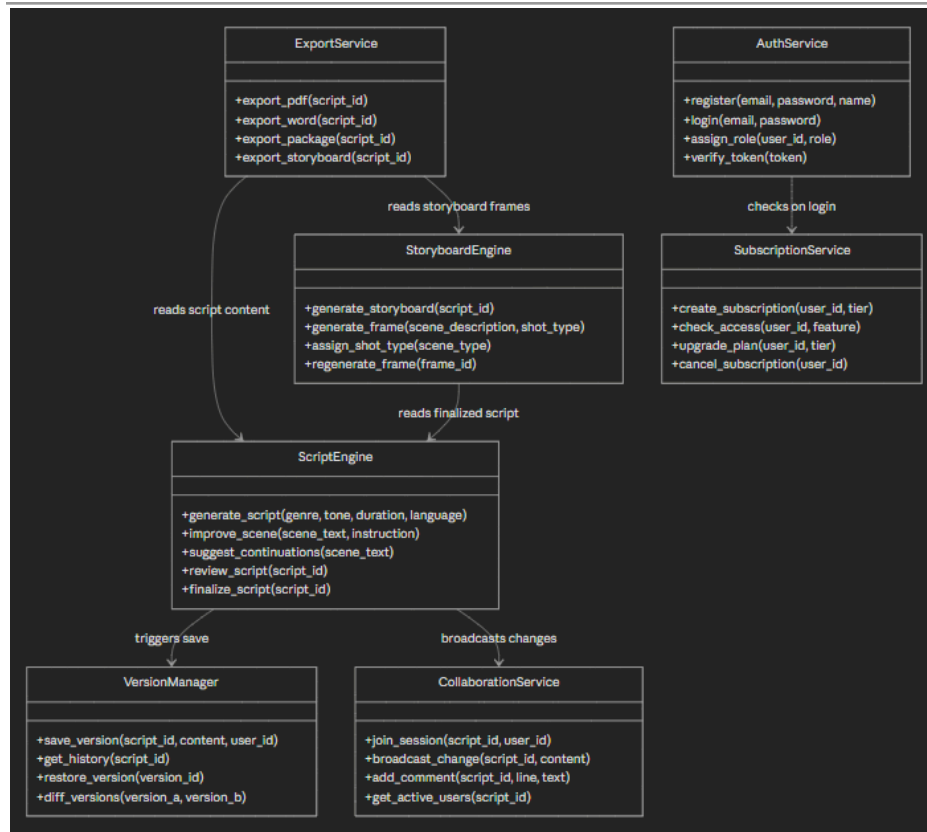
7.2 ER Diagram

Shows the database entities and their relationships: User, Project, Script, Scene, StoryboardFrame, Version, Comment, and Subscription.



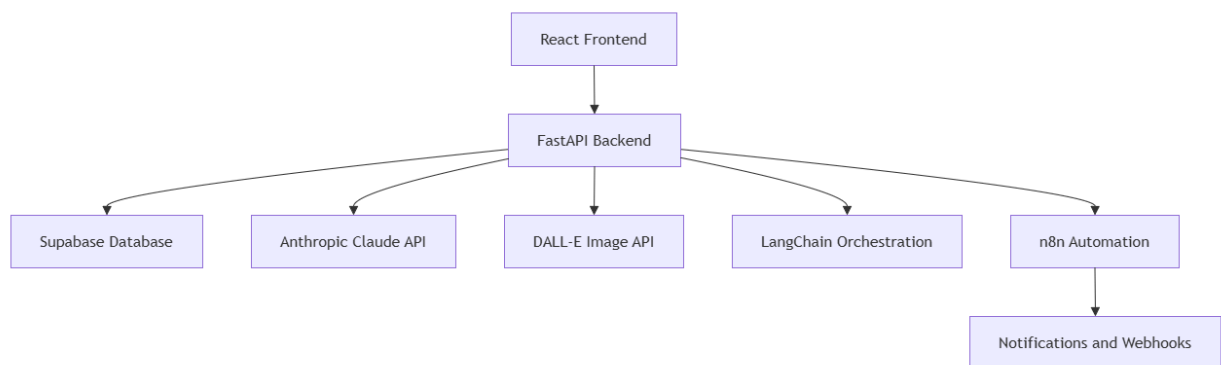
7.3 Class Diagram

Shows the main software classes: ScriptEngine, StoryboardEngine, CollaborationService, VersionManager, ExportService, and their methods and relationships.



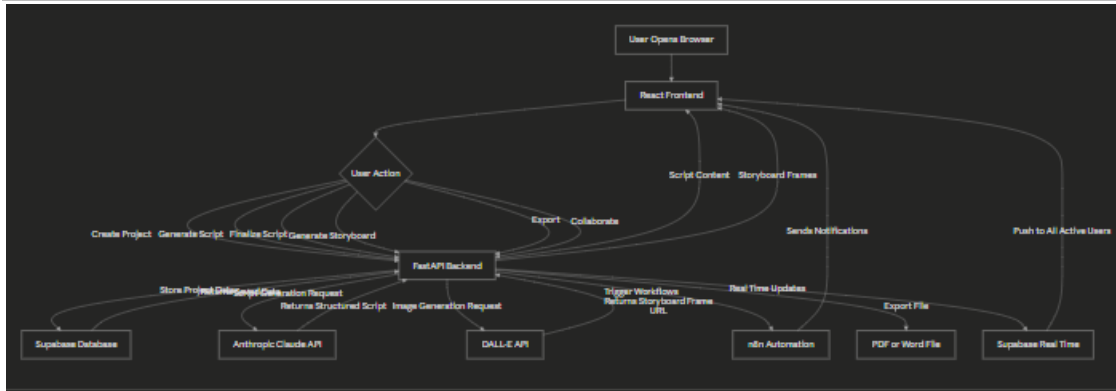
7.4 System Architecture Diagram

Shows the complete system layers: React frontend, FastAPI backend, Supabase database, AI APIs, and n8n automation connected through a central API gateway.



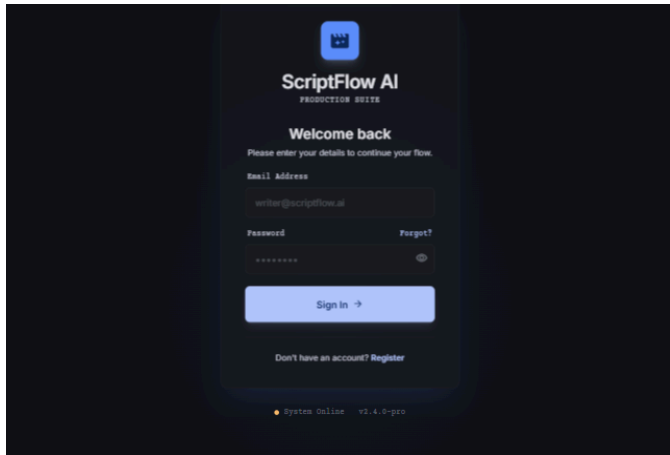
7.5 Data Flow Diagram

Shows how data moves through the system from user input through AI processing to final export output.

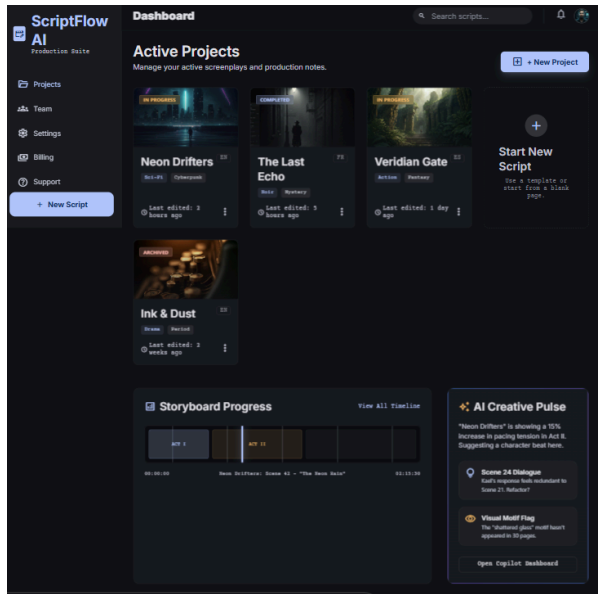


7.6 UI Screens

The following screens form the complete user interface of the system. UI mockups to be inserted here after design generation.



* Screen 1: Login and Registration Page



* Screen 2: Main Dashboard with Project Cards

ScriptFlow AI Drafts Timeline Breakdown Storyboards

STEP 1 OF 3 Project Details

Start a New Script

Establish the core identity of your production. These details will help the AI assist with pacing and structure.

PROJECT TITLE
e.g. Neon Horizon

GENRE
Sci-Fi Noir

DURATION (MINUTES)
110

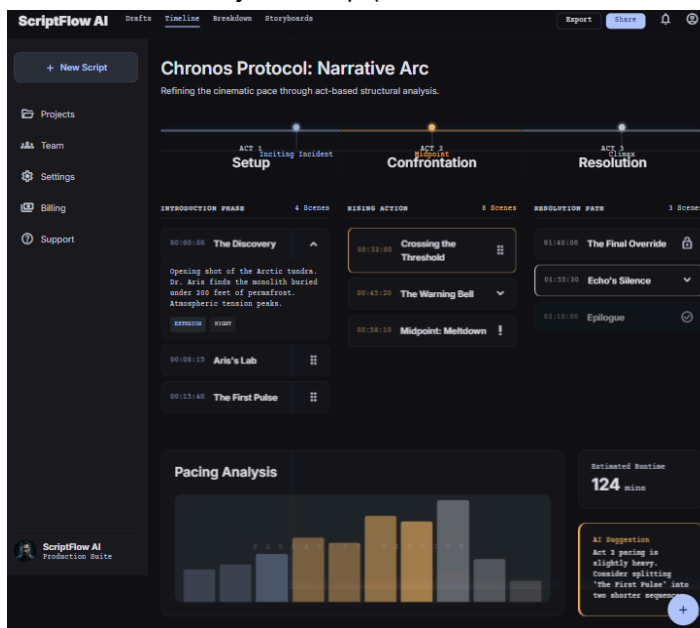
TONE
Dark Humorous Spicy Gritty Surreal Hopeful

TARGET AUDIENCE
e.g. Young Adults 18-25

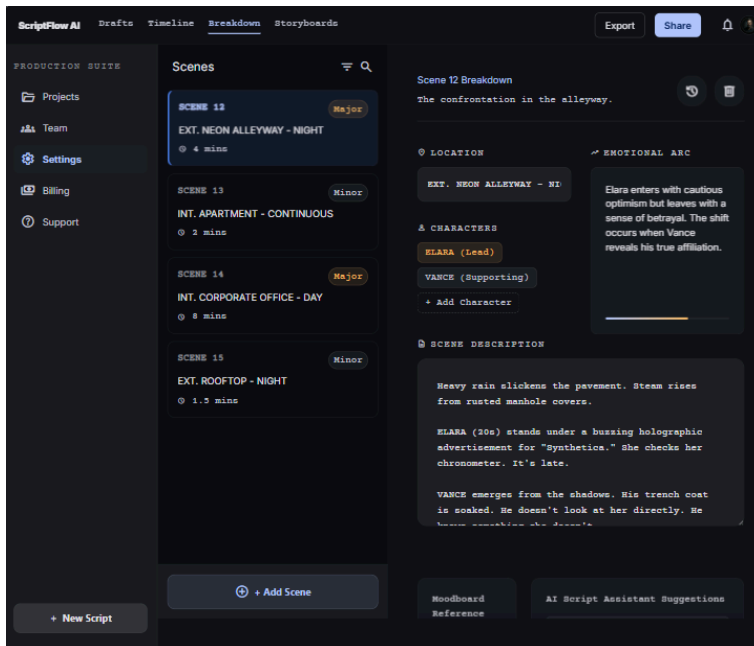
Primary Language EN ES PT

BACK NEXT: STRUCTURE

- * Screen 3: New Project Setup (Genre, Tone, Duration, Language)

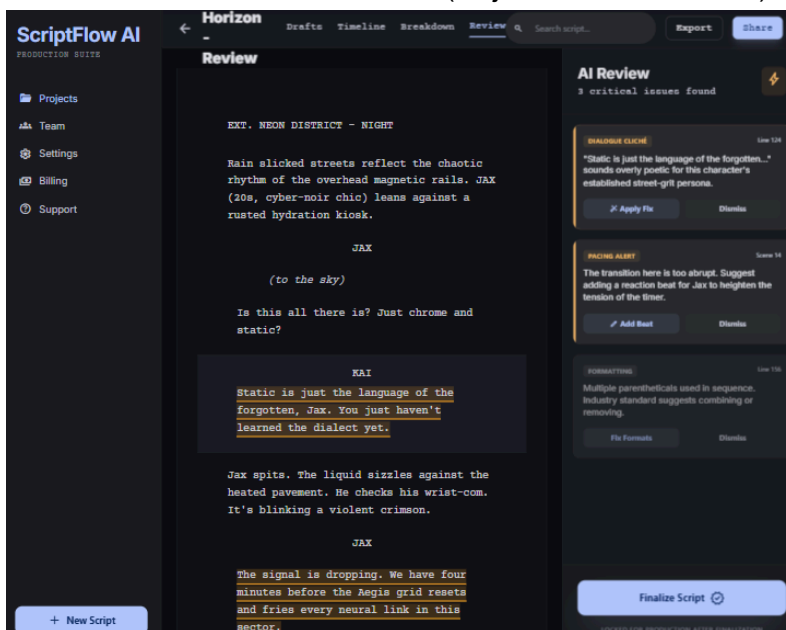


- * Screen 4: Three Act Structure View with Time Allocation



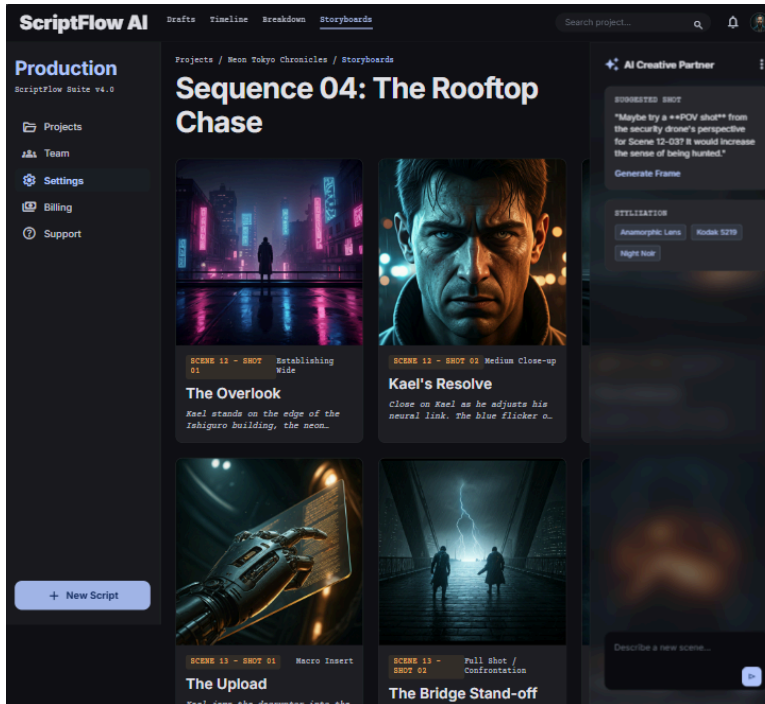
*

* Screen 5: Scene Breakdown Editor (Major and Minor Scenes)



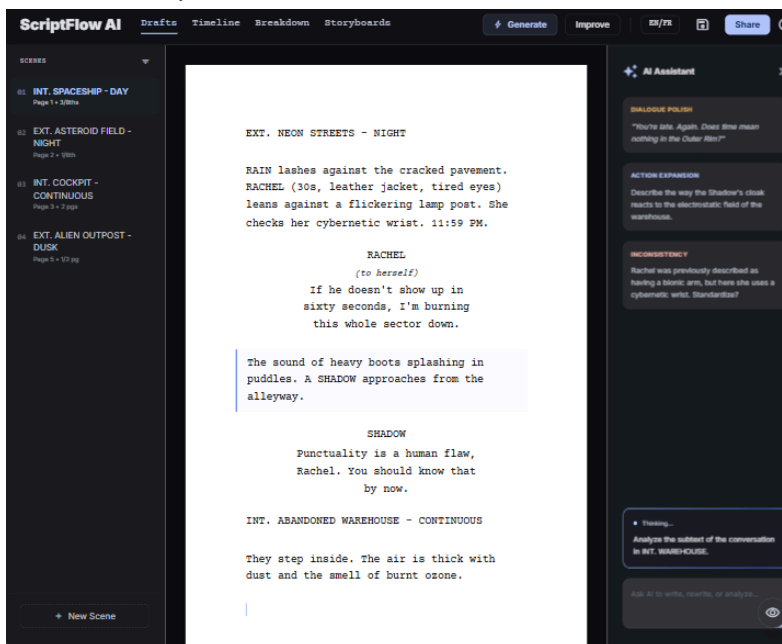
*

Screen 6: Script Editor with AI Assistance Panel



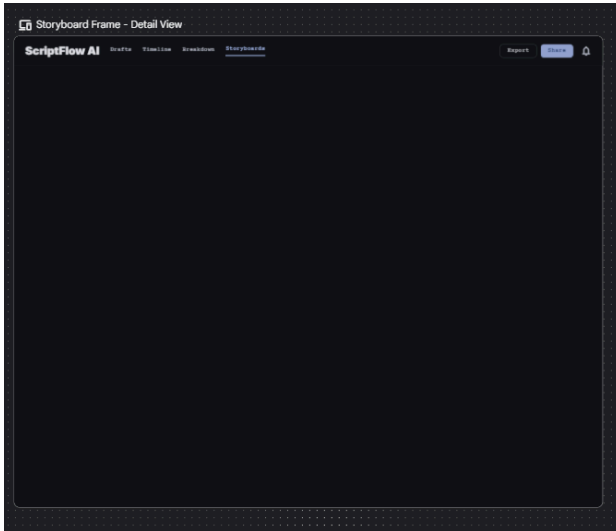
*

* Screen 7: Script Review and Finalization Screen



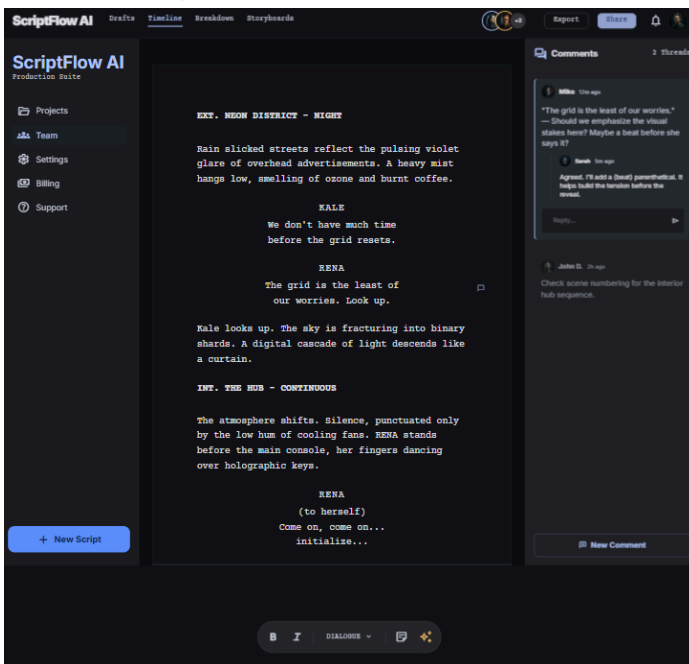
*

* Screen 8: Storyboard View with Frame Grid



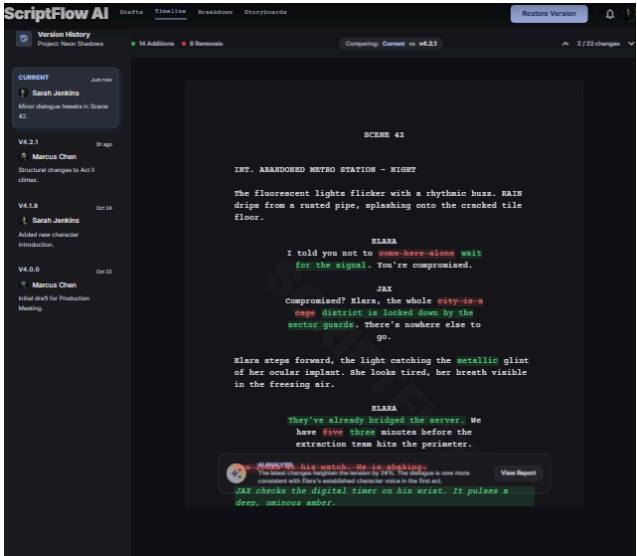
*

* Screen 9: Storyboard Frame Detail and Edit View

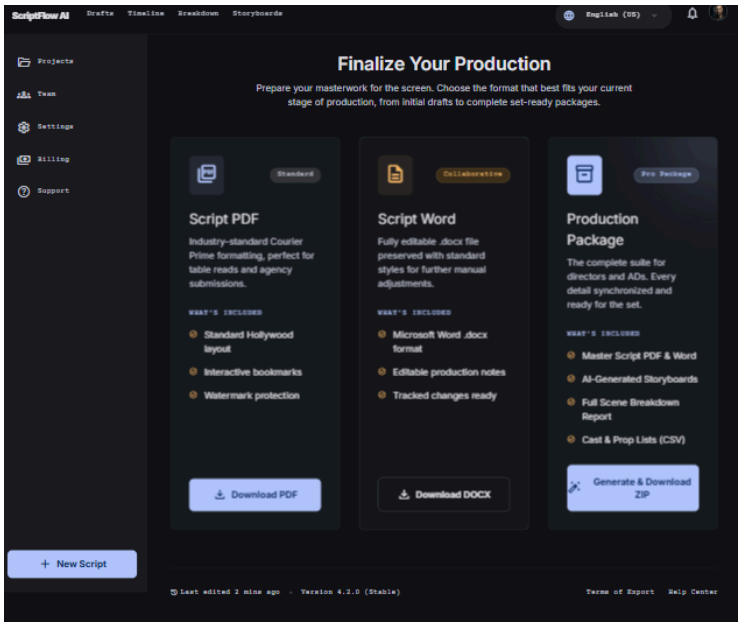


*

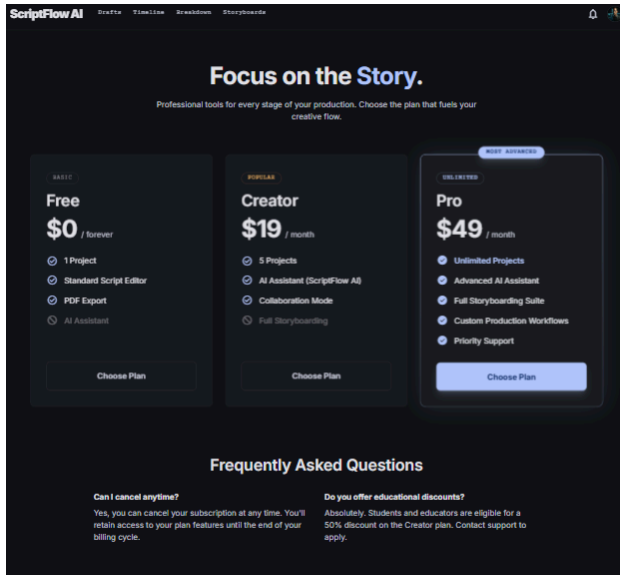
* Screen 10: Collaboration View with Live Cursors and Comments



*
* Screen 11: Version History and Restore View

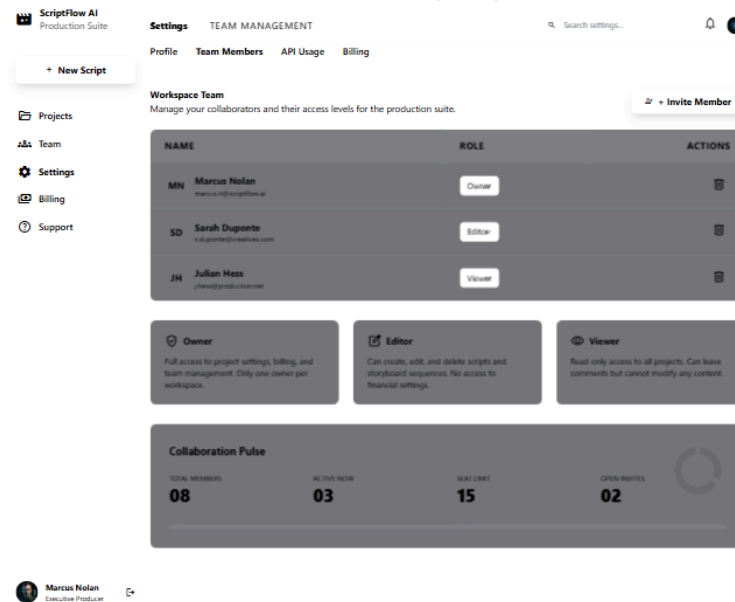


*
* Screen 12: Export Options Screen



*

* Screen 13: Subscription and Pricing Page



*

* Screen 14: Settings and Team Management

8. Time Schedule

The project runs across 12 weeks divided into three monthly phases. A learning module runs in parallel with each build task in the first one to two hours of every workday. Written progress updates are delivered every Friday covering what was completed, what is next, and any blockers.

Wk	Month	Build Focus	Deliverable	Learning Module
1	Month 1	Project Setup	Project structure, database schema, user authentication working	FastAPI and project architecture
2	Month 1	Script Editor UI	Screenplay editor interface built and styled in browser	React components and state
3	Month 1	AI Script Generation	Genre input, three act structure, scene time allocation working	Claude API and prompt engineering
4	Month 1	AI Assistance Mode	Full generation, improvement, and suggestion modes live. Bilingual output working.	LangChain chains and agents
5	Month 2	Storyboard Engine	AI reads script and generates storyboard frames per scene	DALL-E API and image generation
6	Month 2	Storyboard UI	Visual frame grid with regeneration, shot labels, and camera notes	Advanced React state patterns
7	Month 2	Collaboration	Real time editing, inline comments, role based access active	Supabase real time and auth
8	Month 2	Version History	Full version tracking with restore points and diff view	PostgreSQL versioning strategies
9	Month 3	Export System	PDF and Word script export. Combined production package PDF.	PDF generation in Python
10	Month 3	Style Training	AI fine tuned on Baakhapaa voice via prompt engineering	Prompt engineering and fine tuning
11	Month 3	Subscription System	Free, Creator, and Pro tiers live with payment integration	Payment gateway integration
12	Month 3	Testing and Launch	Full testing, bug fixes, live deployment, handover session	Testing and deployment basics

9. Deliverables and Expected Outcomes

9.1 System Deliverables

1. A fully functional AI screenplay generation system with genre definition, three-act structure with 33% time allocation per act, scene-level breakdown, and bilingual AI writing assistance
2. An automated storyboard generator producing visual frames with shot type, camera notes, and scene descriptions for every scene
3. A real-time collaboration system with version history, diff view, inline comments, and role-based access
4. A one-click export system producing scripts in PDF and Word format and a combined production package
5. A subscription platform with Free, Creator, and Pro tiers for external creators

9.2 Technical Deliverables

1. Complete source code repository with inline documentation
2. System architecture document and API reference guide
3. Live deployment on Vercel and Railway with full environment configuration
4. Handover walkthrough session and written handover notes

9.3 Expected Outcomes

- * Content preproduction time reduced by at least 60 percent compared to the current fully manual process
- * A live subscription-based platform accessible to external Nepali and international creators by the end of Month 3
- * A scalable system architecture that supports future modules, including video analysis and product tagging, without rebuilding the core platform

10. References

All references follow APA 7th Edition citation style.

Anthropic. (2024). Claude API documentation. Anthropic. <https://docs.anthropic.com>

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

Chase, H. (2022). LangChain: Building applications with LLMs through composability [GitHub repository]. GitHub. <https://github.com/langchain-ai/langchain>

Eloundou, T., Manning, S., Mishkin, P., & Rock, D. (2023). GPTs are GPTs: An early look at the labor market impact potential of large language models. *arXiv*. <https://arxiv.org/abs/2303.10130>

FastAPI. (2023). FastAPI documentation: High performance, easy to learn, fast to code. Tiangolo. <https://fastapi.tiangolo.com>

OpenAI. (2023). GPT-4 technical report. OpenAI. <https://openai.com/research/gpt-4>

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 9. <https://openai.com/research/language-unsupervised>

Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., & Sutskever, I. (2021). Zero-shot text-to-image generation. *Proceedings of the 38th International Conference on Machine Learning*, 139, 8821-8831.

Supabase. (2023). Supabase documentation: The open source Firebase alternative. Supabase Inc. <https://supabase.com/docs>

Vercel. (2024). Vercel platform documentation. Vercel Inc. <https://vercel.com/docs>